

Testanleitung

Testphilosophie

Das Geef.Sdk ist bewusst **LLM-agnostisch** — es hat keine Abhängigkeit auf ein KI-SDK. Das macht Unit-Tests einfach: Sie implementieren die Provider-Interfaces mit deterministischer Logik und testen die Orchestrierung isoliert.

Test-Doubles (Delegate-Pattern)

Für Tests empfehlen wir einfache Delegate-basierte Implementierungen:

```

private sealed class DelegateGrounding(Func<string, GroundingResult> fn) : IGroundingStep
{
    public Task<GroundingResult> RunAsync(string input, CancellationToken ct = default)
    => Task.FromResult(fn(input));
}

private sealed class DelegateExecution(
    Func<IRunContext, CancellationToken, Task<ExecutionResult>> fn) : IExecutionStep
{
    public Task<ExecutionResult> RunAsync(IRunContext ctx, CancellationToken ct = default)
    => fn(ctx, ct);
}

private sealed class DelegateReviewer(
    string name,
    Func<IRunContext, CancellationToken, Task<ReviewResult>> fn) : IReviewer
{
    public string Name => name;
    public Task<ReviewResult> ReviewAsync(IRunContext ctx, CancellationToken ct = default)
    => fn(ctx, ct);
}

private sealed class DelegateFinalizer<T>(
    Func<IRunContext, Task<FinalizeResult<T>>> fn) : IFinalizer<T>
{
    public Task<FinalizeResult<T>> FinalizeAsync(IRunContext ctx, CancellationToken ct = default)
    => fn(ctx);
}

```

Unit-Tests

Context testen

```
[Fact]
public void Set_returns_new_context_with_value()
{
    var key = new ContextKey<string>("test");
    var ctx = new RunContext();

    var updated = ctx.Set(key, "hello");

    Assert.False(ctx.Contains(key));           // Original unverändert
    Assert.Equal("hello", updated.GetRequired(key));
}

[Fact]
public void GetRequired_throws_when_key_missing()
{
    var key = new ContextKey<int>("missing");
    var ctx = new RunContext();

    Assert.Throws<KeyNotFoundException>(() => ctx.GetRequired(key));
}
```

Pipeline-Happy-Path

```
[Fact]
public async Task Pipeline_succeeds_on_first_attempt()
{
    var key = new ContextKey<string>("artifact");

    var pipeline = Geef.CreatePipeline<string>()
        .UseGrounding(new DelegateGrounding(input =>
            new GroundingResult { Context = new RunContext() }))
        .UseExecution(new DelegateExecution((ctx, _) =>
            Task.FromResult(new ExecutionResult
            {
                UpdatedContext = ctx.Set(key, "generated")
            })))
        .AddReviewer(new DelegateReviewer("OK", (_, _) =>
            Task.FromResult(new ReviewResult
            {
                ReviewerName = "OK",
                Decision = ReviewDecision.Approved,
                Duration = TimeSpan.FromMilliseconds(1)
            })))
        .UseFinalizer(new DelegateFinalizer<string>(ctx =>
            Task.FromResult(new FinalizeResult<string>
            {
                Output = ctx.GetRequired(key),
                FinalContext = ctx
            })))
        .Build();

    var result = await pipeline.RunAsync("test");

    Assert.True(result.Success);
    Assert.Equal("generated", result.Output);
    Assert.Equal(1, result.TotalIterations);
}
```

Retry-Verhalten testen

```
[Fact]
public async Task Pipeline_retries_and_succeeds_on_second_iteration()
{
    var key = new ContextKey<string>("artifact");
    var callCount = 0;

    var pipeline = Geef.CreatePipeline<string>()
        .UseGrounding(new DelegateGrounding(_ =>
            new GroundingResult { Context = new RunContext() }))
        .UseExecution(new DelegateExecution((ctx, _) =>
        {
            callCount++;
            return Task.FromResult(new ExecutionResult
            {
                UpdatedContext = ctx.Set(key, $"v{callCount}")
            });
        }))
        .AddReviewer(new DelegateReviewer("Gate", (ctx, _) =>
        {
            var value = ctx.GetRequired(key);
            var approved = value == "v2";
            return Task.FromResult(new ReviewResult
            {
                ReviewerName = "Gate",
                Decision = approved ? ReviewDecision.Approved : ReviewDecision.Reject
            });
        }))
        .UseFinalizer(new DelegateFinalizer<string>(ctx =>
            Task.FromResult(new FinalizeResult<string>
            {
                Output = ctx.GetRequired(key),
            }));
}
```

```
        FinalContext = ctx
    })))
    .Build();

var result = await pipeline.RunAsync("input");

Assert.True(result.Success);
Assert.Equal("v2", result.Output);
Assert.Equal(2, result.TotalIterations);
}
```

PreviousFindings testen

```
[Fact]
public async Task Execution_receives_previous_findings_on_retry()
{
    IReadOnlyList<Finding>? capturedFindings = null;
    var callCount = 0;
    var key = new ContextKey<string>("out");

    var pipeline = Geef.CreatePipeline<string>()
        .UseGrounding(new DelegateGrounding(_ =>
            new GroundingResult { Context = new RunContext() }))
        .UseExecution(new DelegateExecution((ctx, _) =>
        {
            callCount++;
            if (callCount > 1)
                ctx.TryGet(GeefKeys.PreviousFindings, out capturedFindings);
            return Task.FromResult(new ExecutionResult
            {
                UpdatedContext = ctx.Set(key, $"v{callCount}")
            });
        })))
        .AddReviewer(new DelegateReviewer("R", (ctx, _) =>
        {
            var ok = ctx.GetRequired(key) == "v2";
            return Task.FromResult(new ReviewResult
            {
                ReviewerName = "R",
                Decision = ok ? ReviewDecision.Approved : ReviewDecision.Rejected,
                Findings = ok
                    ? Array.Empty<Finding>()
                    : new[] { new Finding { ReviewerName = "R", Fingerprint = "fp1",
Message = "Fix" } },
                Duration = TimeSpan.FromMilliseconds(1)
            });
        })))
        .UseFinalizer(new DelegateFinalizer<string>(ctx =>
            Task.FromResult(new FinalizeResult<string> { Output = "done", FinalContext = ctx })))
        .Build();

    await pipeline.RunAsync("x");
}
```

```
Assert.NotNull(capturedFindings);  
Assert.Single(capturedFindings!);  
Assert.Equal("fp1", capturedFindings![0].Fingerprint);  
}
```

Konvergenz-Fehler testen

```
[Fact]
public async Task Pipeline_throws_ConvergenceFailedException_at_max_iterations()
{
    var pipeline = Geef.CreatePipeline<string>()
        .UseGrounding(new DelegateGrounding(_ =>
            new GroundingResult { Context = new RunContext() }))
        .UseExecution(new DelegateExecution((ctx, _) =>
            Task.FromResult(new ExecutionResult { UpdatedContext = ctx })))
        .AddReviewer(new DelegateReviewer("AlwaysFail", (_, _) =>
            Task.FromResult(new ReviewResult
            {
                ReviewerName = "AlwaysFail",
                Decision = ReviewDecision.Rejected,
                Findings = new[] { new Finding
                {
                    ReviewerName = "AlwaysFail",
                    Fingerprint = "always",
                    Message = "Immer falsch"
                }
                },
                Duration = TimeSpan.FromMilliseconds(1)
            })))
        .UseFinalizer(new DelegateFinalizer<string>(ctx =>
            Task.FromResult(new FinalizeResult<string> { Output = "x", FinalContext =
ctx })))
        .UseConvergencePolicy(new DefaultConvergencePolicy
        {
            MaxIterations = 3,
            StagnationThreshold = 99 // Stagnation deaktivieren
        })
        .Build();

    var ex = await Assert.ThrowsAsync<ConvergenceFailedException>(
        () => pipeline.RunAsync("input"));

    Assert.Equal(ConvergenceDecision.StopMaxAttemptsReached, ex.Reason);
    Assert.Equal(3, ex.History.Count);
}
```

Events testen

```
[Fact]
public async Task Events_fire_in_correct_order()
{
    var events = new List<string>();
    var key = new ContextKey<string>("out");

    var pipeline = Geef.CreatePipeline<string>()
        .UseGrounding(new DelegateGrounding(_ =>
            new GroundingResult { Context = new RunContext() }))
        .UseExecution(new DelegateExecution((ctx, _) =>
            Task.FromResult(new ExecutionResult { UpdatedContext = ctx.Set(key, "x")
        }))))
        .AddReviewer(new DelegateReviewer("R", (_, _) =>
            Task.FromResult(new ReviewResult
            {
                ReviewerName = "R",
                Decision = ReviewDecision.Approved
            })))
        .UseFinalizer(new DelegateFinalizer<string>(ctx =>
            Task.FromResult(new FinalizeResult<string> { Output = "done", FinalContext = ctx })))
        .ConfigureEvents(e =>
        {
            e.OnPipelineStarted = _ => { events.Add("Start"); return Task.CompletedTask; };
            e.OnGroundingStarted = _ => { events.Add("Ground"); return Task.CompletedTask; };
            e.OnExecutionStarted = _ => { events.Add("Exec"); return Task.CompletedTask; };
            e.OnEvaluationApproved = _ => { events.Add("Approved"); return Task.CompletedTask; };
            e.OnPipelineCompleted = _ => { events.Add("Done"); return Task.CompletedTask; };
        })
        .Build();

    await pipeline.RunAsync("input");

    Assert.Equal(new[] { "Start", "Ground", "Exec", "Approved", "Done" }, events);
}
```

Cancellation testen

```
[Fact]
public async Task Cancellation_propagates()
{
    using var cts = new CancellationTokenSource();

    var pipeline = Geef.CreatePipeline<string>()
        .UseGrounding(new DelegateGrounding(_ =>
        {
            cts.Cancel(); // Sofort canceln
            return new GroundingResult { Context = new RunContext() };
        }))
        .UseExecution(new DelegateExecution(async (ctx, ct) =>
        {
            await Task.Delay(5000, ct); // Sollte abbrechen
            return new ExecutionResult { UpdatedContext = ctx };
        }))
        .AddReviewer(new DelegateReviewer("R", (_, _) =>
            Task.FromResult(new ReviewResult { ReviewerName = "R", Decision = ReviewDecision.Approved })))
        .UseFinalizer(new DelegateFinalizer<string>(ctx =>
            Task.FromResult(new FinalizeResult<string> { Output = "x", FinalContext = ctx })))
        .Build();

    await Assert.ThrowsAsync<OperationCanceledException>(
        () => pipeline.RunAsync("input", cts.Token));
}
```

Integrationstests

Für End-to-End-Tests mit echten LLM-Anbindungen:

```
[Trait("Category", "Integration")]

public class LlmIntegrationTests
{
    [Fact]
    public async Task Real_llm_pipeline_converges()
    {
        var pipeline = Geef.CreatePipeline<string>()
            .UseGrounding(new RealRagGrounding(config))
            .UseExecution(new OpenAiExecution(apiKey))
            .AddReviewer(new OpenAiReviewer(apiKey))
            .UseFinalizer(new MarkdownFinalizer())
            .UseConvergencePolicy(new DefaultConvergencePolicy { MaxIterations = 5 })
            .Build();

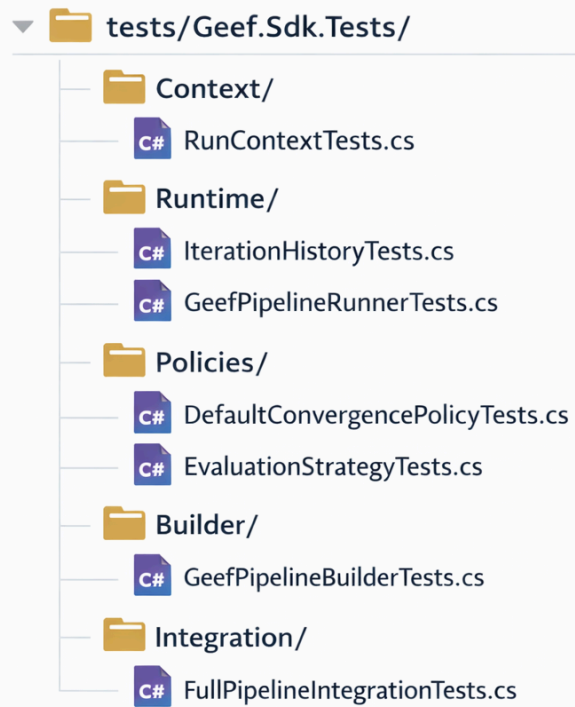
        var result = await pipeline.RunAsync("Erkläre Dependency Injection");

        Assert.True(result.Success);
        Assert.True(result.Output.Length > 200);
    }
}
```

Filtern mit:

```
dotnet test --filter "FullName~IntegrationTests"
```

Test-Projektstruktur



Tests ausführen

```
# Alle Tests
dotnet test tests/Geef.Sdk.Tests/

# Nur Unit-Tests
dotnet test tests/Geef.Sdk.Tests/ --filter "Category!=Integration"

# Nur Integrationstests
dotnet test tests/Geef.Sdk.Tests/ --filter "FullName~IntegrationTests"

# Verbose
dotnet test tests/Geef.Sdk.Tests/ -v detailed
```